

מקיף ז' - הקריה, אשדוד

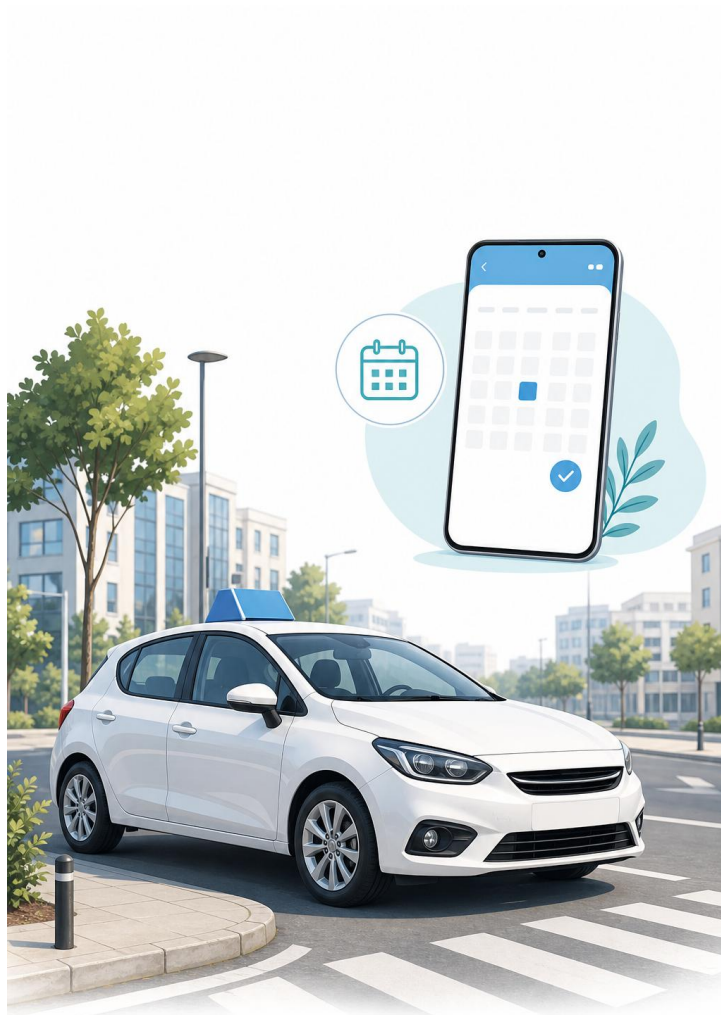


מגמת הנדסת תוכנה 5 יח"ל

Driver Moodle

מערכת לניהול שיעורי נהיגה

פרויקט גמר - שירותי רשת, בסיס נתונים ותכנות אסינכרוני



שם התלמיד: אלי רפפורט

תעודת זהות: 327588232

שם המנחה: אולגה גרדלמן

תאריך הגשה: יוני 2026

תוכן עניינים

3	בדיקת דרישות המחונן
4	פרק 1: מבוא ותרחישי שימוש
4	למה בחרתי בפרויקט
4	מי משתמש במערכת
4	Use Case
6	פרק 2: בסיס הנתונים
6	טבלאות מרכזיות
6	קשרים בין הטבלאות
7	קשרים לוגיים נוספים
8	גישה לבסיס הנתונים
10	פרק 3: צד השרת
10	מבנה כללי
11	פרויקט Model
12	פרויקט ViewDB
13	פרויקט BusinessLogic
13	שירות WCF
13	קובצי App.config והגדרות הרשת
14	פרק 4: צד הלקוח
14	לקוח WPF
15	Binding, XAML ו-ValueConverter
15	לקוח Web
15	לקוח MAUI / Android
15	Async / Tasks
16	פרק 5: תהליכים מרכזיים ואבטחה
16	קביעת שיעור
16	התראות
17	תשלומים ודוחות
17	בדיקות קלט ואבטחה
18	פרק 6: מדריך הפעלה ובדיקות
18	הפעלה קצרה
18	בדיקות שביצעתי
18	הערה לגבי מסכי הקורסים
19	פרק 7: נספחים
19	קבצים שכדאי לפתוח בזמן הבחינה
19	סיכום קצר

בדיקת דרישות המבחון

כדי שיהיה קל לבדוק את התיק, חילקתי בין דרישות החובה לבין דרישות הבחירה. בכל שורה יש קישור שמוביל ישירות להסבר המתאים בתוך המסמך.

מספר	סוג	דרישה	איפה מימשי	מעבר לפירוט
1	חובה	✓ מערכת לקוח-שרת	שלושה לקוחות פונים לאותו שירות WCF	לפירוט
2	חובה	✓ בסיס נתונים יחסי	מסד Access עם טבלאות וקשרים	לפירוט
3	חובה	✓ שכבות בפרויקט	Model, ViewDB, BusinessLogic ושירות	לפירוט
4	חובה	✓ פעולות CRUD	שליפה, הוספה, עדכון ומחיקה דרך ViewDB	לפירוט
5	חובה	✓ לקוח WPF	יישום למחשב הביתי	לפירוט
6	חובה	✓ לקוח Web	דפי Razor Pages בדפדפן	לפירוט
7	חובה	✓ לקוח MAUI	יישום Android לפעולות קצרות מהטלפון	לפירוט
8	חובה	✓ Use Case והרשאות	תלמיד, מורה ומנהל	לפירוט
9	בחירה	✓ ירושה ומחלקות גנריות	Base, BaseDB ורשימות אובייקטים	לפירוט
9	בחירה	✓ ניהול מספר משתמשים	הפרדה לפי מזהה משתמש וסוג משתמש	לפירוט
10	בחירה	✓ IValueConverter ו-XAML	המרת מידע לתצוגה בלקוח WPF	לפירוט
10	בחירה	✓ Validation	בדיקות קלט לפני שמירה	לפירוט
10	בחירה	✓ שירות חיצוני	WCF מחבר בין הלקוחות לשרת	לפירוט
10	בחירה	✓ SQL Injection	OleDbParameter במקום חיבור טקסט	לפירוט
10	בחירה	✓ Async / Tasks	קריאות אסינכרוניות בלקוח MAUI	לפירוט

פרק 1: מבוא ותרשישי שימוש

למה בחרתי בפרויקט

בחרתי לבנות את Driver Moodle בגלל החוויה האישית שלי בזמן שלמדתי נהיגה. לא תמיד היה לי קל לקבוע שיעור. לפעמים קבענו דרך הודעות, לפעמים שעה השתנתה, ולא תמיד היה ברור כמה שיעורים כבר עשיתי ומה נשאר לשלם. רציתי לרכז את הדברים במקום אחד.

המערכת היא מערכת לקוח-שרת לניהול שיעורי נהיגה. הנתונים לא נשמרים בתוך מסך מסוים. כל לקוח שולח בקשות לשירות הרשת, ורק השרת עובד מול בסיס הנתונים.

מי משתמש במערכת

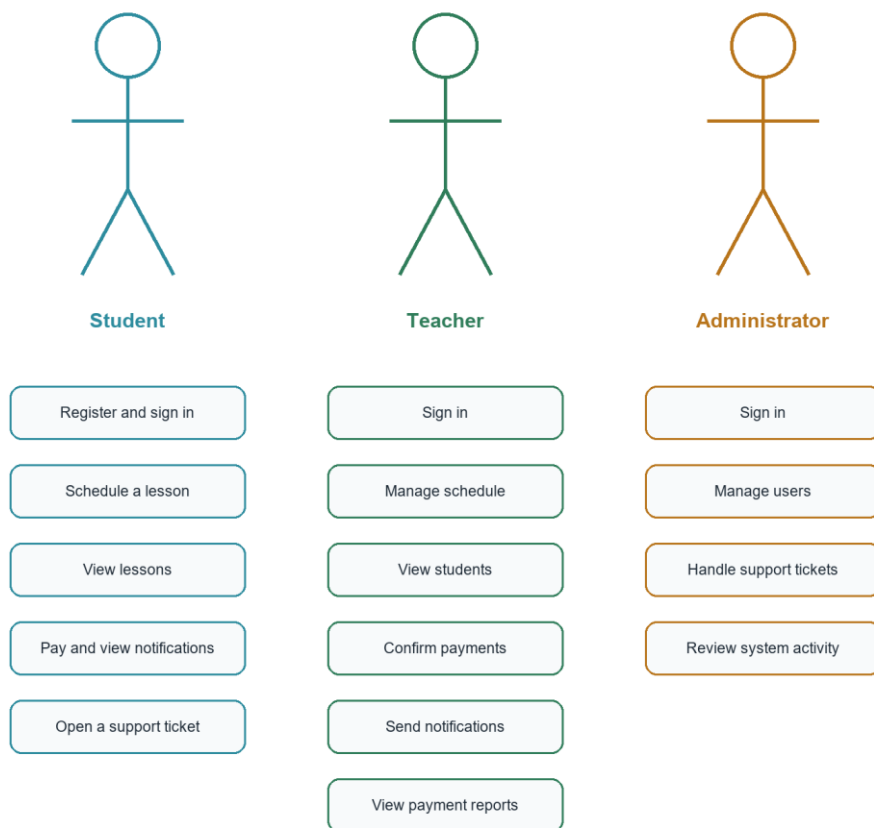
- תלמיד: קובע שיעורים, צופה בשיעורים, רואה תשלומים, מקבל התראות ושולח פנייה.
- מורה: מנהל לוח זמנים, רואה תלמידים, מאשר תשלומים, שולח התראות ורואה דוחות.
- מנהל: מנהל משתמשים ומטפל בפניות תמיכה.

Use Case

חילקתי את הפעולות לפי סוג המשתמש. כך כל משתמש רואה את המסכים שמתאימים לתפקיד שלו, ולא מקבל גישה למסכים שלא שייכים אליו.

Driver Moodle - Use Cases

The main actions are separated by the role of the user



תרשים 1: הפעולות המרכזיות לפי סוג המשתמש.

פרק 2: בסיס הנתונים

בסיס הנתונים נשמר בקובץ UsersDataBase.accdb מסוג Access. בחרתי ב-Access כי הוא מתאים להיקף של פרויקט לימודי, מאפשר לעבוד עם טבלאות וקשרים, וקל להציג אותו בזמן הבחינה.

לא שמרתי את כל המידע בטבלה אחת. הפרדתי בין תלמידים, מורים, שיעורים, תשלומים, התראות ופניות תמיכה. כך אפשר לעדכן נתון אחד בלי לפגוע בנתונים אחרים.

טבלאות מרכזיות

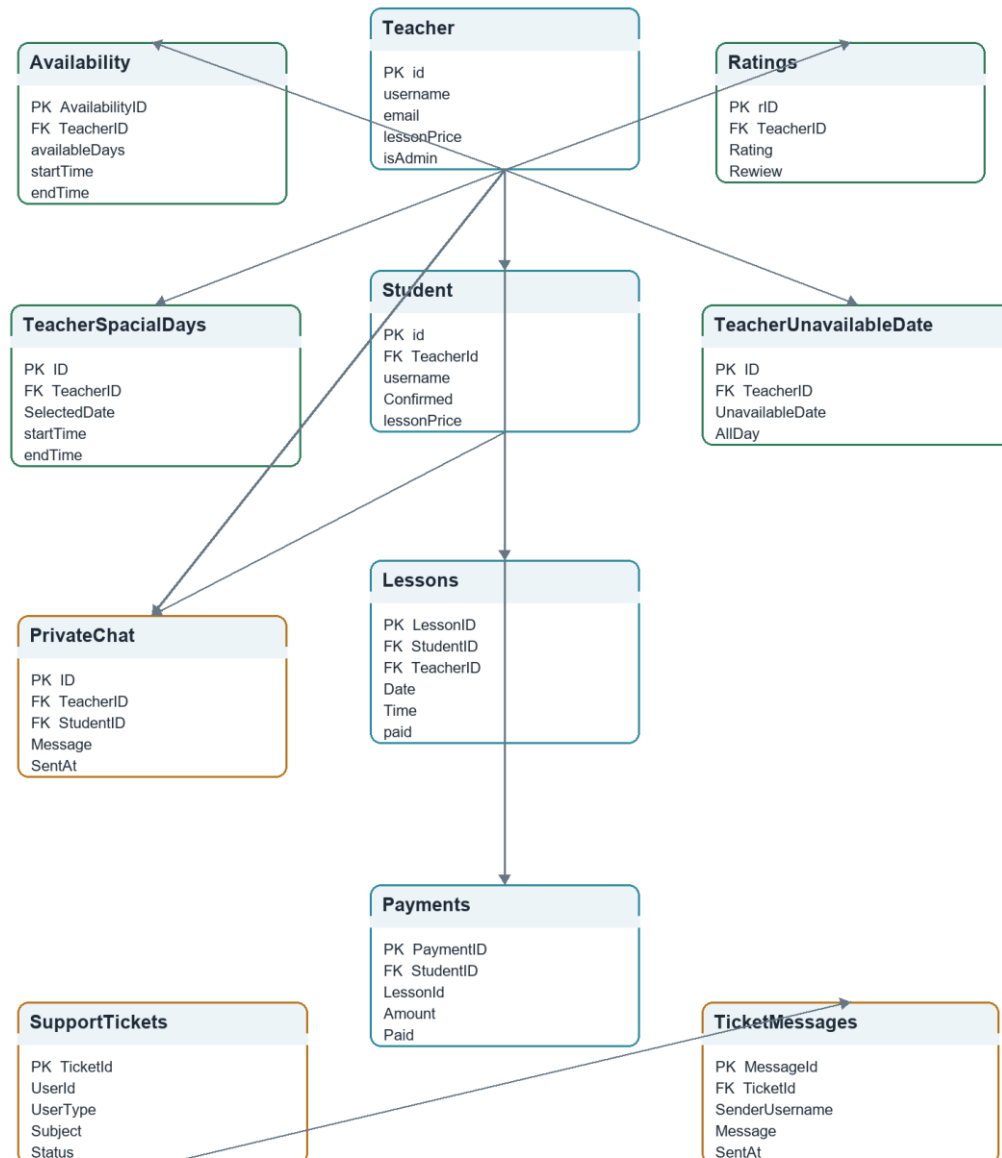
טבלה	מה נשמר בה
Teacher	פרטי מורה, מחיר שיעור, הרשאת מנהל ואמצעי תשלום.
Student	פרטי תלמיד, המורה שלו, מצב אישור ומחיר מותאם.
Lessons	שיעור, תלמיד, מורה, תאריך, שעה ומצב תשלום.
Payments	תשלום, סכום, תלמיד, שיעור ומצב התשלום.
Notifications	שולח, מקבל, כותרת, הודעה ומצב קריאה.
SupportTickets	פניית תמיכה, נושא, תיאור, מצב ועדיפות.
TicketMessages	הודעות שנכתבו בתוך פנייה.

קשרים בין הטבלאות

בתרשים הבא מופיעים הקשרים המרכזיים שנשמרים במסד. לדוגמה, תלמיד מקושר למורה, שיעור מקושר לתלמיד ולמורה, ותשלום מקושר לתלמיד ולשיעור.

Access Relationships - Core Tables

Solid lines represent relationships stored in the Access database



PK = primary key FK = foreign key

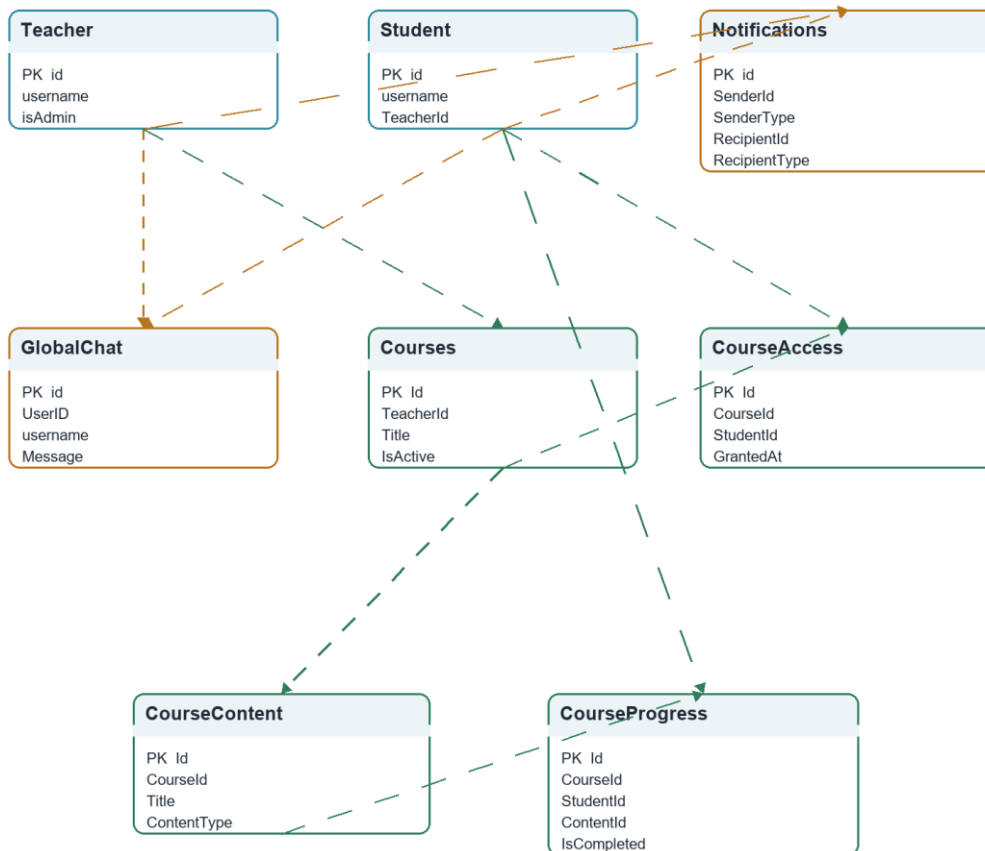
תרשים 2: קשרים מרכזיים במסד הנתונים. הקווים הרציפים מייצגים קשרים שמוגדרים במסד.

קשרים לוגיים נוספים

יש גם טבלאות שבהן הקישור נשמר באמצעות מזהים ובבדק בקוד השירות. לדוגמה, התראות נשמרות לפי מזהה משתמש וסוג משתמש. בתרשים הבא סימנתי קשרים כאלה בקו מקווקו כדי לא להציג אותם כאילו הם קשרי מפתח זר שהוגדרו במסד.

Database Tables - Additional Logical Connections

Dashed lines are identifiers managed by the service code



Why dashed lines?

These tables contain matching IDs, but the repository currently manages their connection in service code instead of an Access foreign-key rule.

תרשים 3: טבלאות וקשרים לוגיים שמנהלים בקוד השירות.

גישה לבסיס הנתונים

הלקוחות לא ניגשים ישירות ל-Access. העבודה עוברת דרך שכבת ViewDB. המחלקה BaseDB מרכזת את החיבור, את מנגנון השליפה ואת מנגנון השמירה. המחלקות האחרות משתמשות בה.

קטע קוד: מנגנוני Select ו-SaveChanges במחלקה BaseDB.

```
protected virtual List<Base> Select(string sqlCommandTxt,
                                     params OleDbParameter[] parameters)
{
    connection.Open();
    command.CommandText = sqlCommandTxt;
    command.Parameters.Clear();
```



```

        command.Parameters.AddRange(parameters);
        reader = command.ExecuteReader();
        ...
    }

protected int SaveChanges(string commandText,
                           params OleDbParameter[] parameters)
{
    cmd.CommandText = commandText;
    cmd.Parameters.AddRange(parameters);
    connection.Open();
    return cmd.ExecuteNonQuery();
}

```

באמצעות שני המנגנונים האלה מתבצעות פעולות CRUD: שליפה, הוספה, עדכון ומחיקה. הפקודה משתנה לפי הפעולה, אבל פתיחת החיבור והטיפול בפרמטרים נשארים במקום אחד.

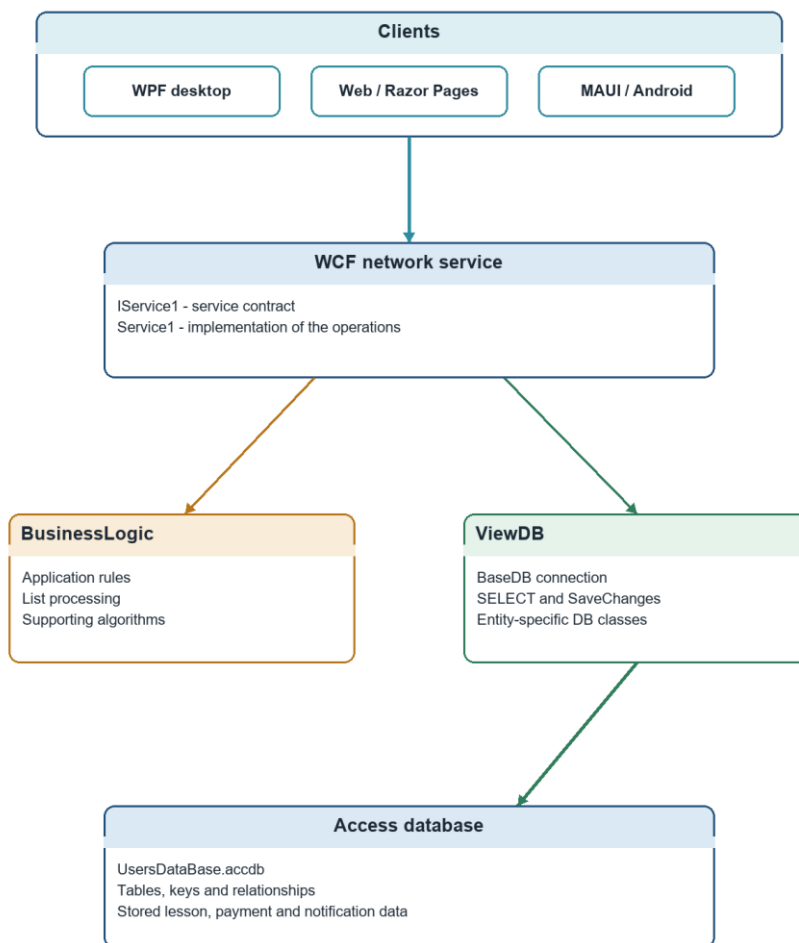
פרק 3: צד השרת

מבנה כללי

צד השרת מחולק לפרויקטים נפרדים. לכל פרויקט יש תפקיד ברור. החלוקה עוזרת לשמור על סדר ומונעת מצב שבו מסך הלקוח מכיל גם קוד של בסיס הנתונים.

Driver Moodle - System Architecture

A clear view of clients, network service, application layers and database



Main request path

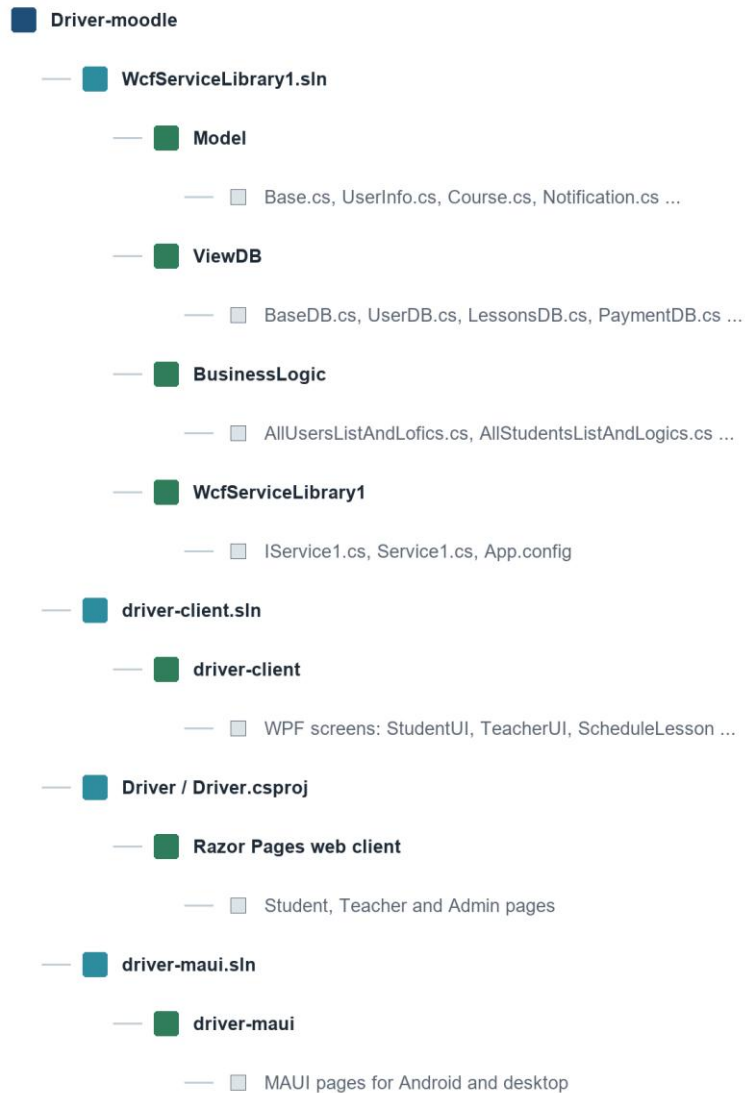
Client -> IService1 -> Service1 -> ViewDB -> Access database

The clients do not connect directly to the database.

תרשים 4: מבנה המערכת. כל הלקוחות עובדים מול אותו שירות.

Driver Moodle - Solution Structure

Readable project tree based on the real folders in the repository



Each server project has a separate responsibility. The three clients use the same WCF service.

תרשים 5: מבנה הפתרון לפי התיקיות והפרויקטים האמיתיים בפרויקט.

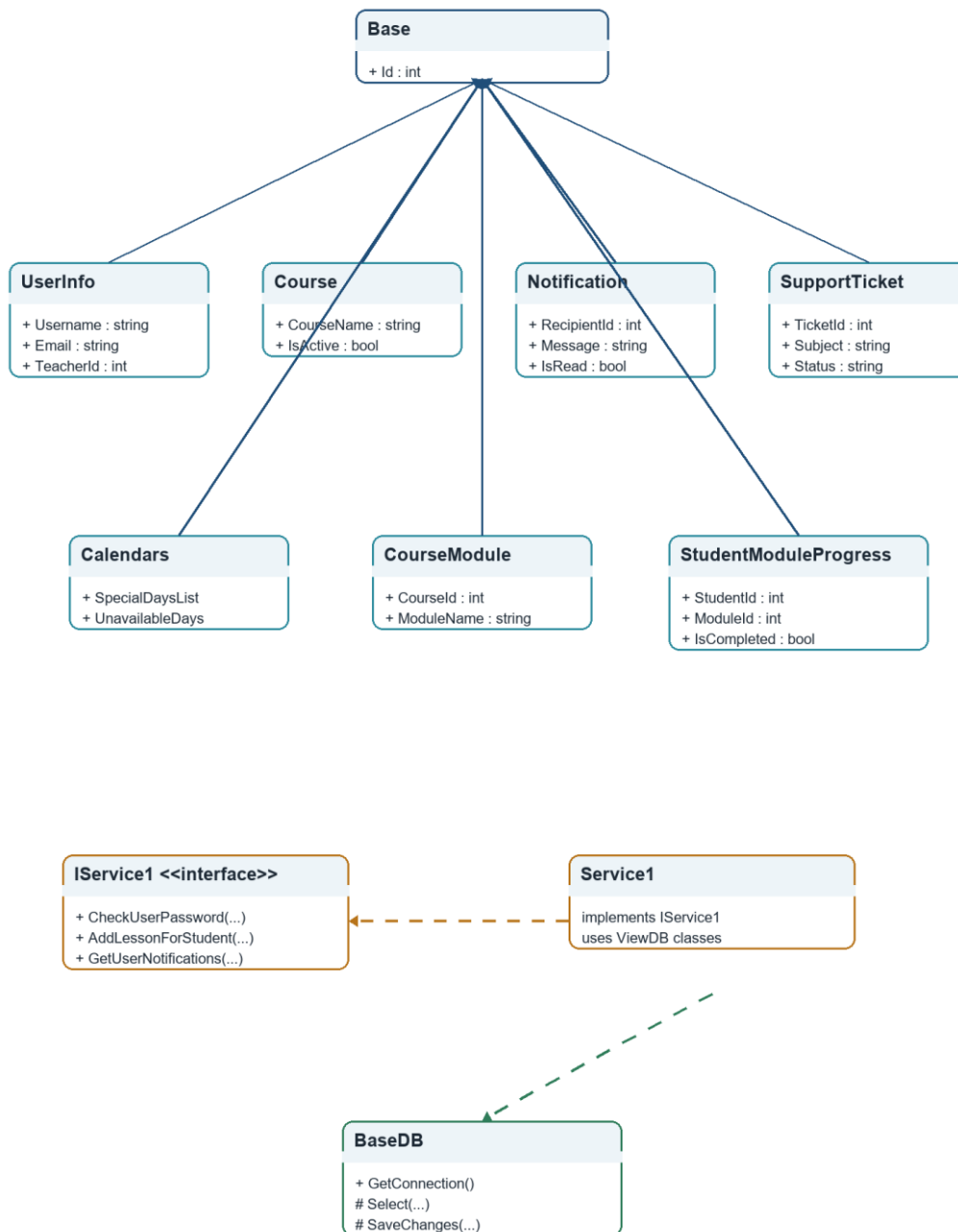
פרויקט Model

בפרויקט Model נמצאות מחלקות שמייצגות מידע שעובר בין הלקוח לשרת ובחזרה. רוב המחלקות מכילות Properties. לדוגמה, UserInfo מייצגת משתמש, Notification מייצגת התראה, ו-Course מייצגת קורס.

מחלקות מרכזיות יורשות מהמחלקה Base. במחלקה Base נמצא Id משותף. כך לכל אובייקט יש מזהה שאפשר לקשר לרשומה המתאימה במסד הנתונים.

UML Class Diagram - Main Classes

Inheritance and implementation relationships used in the project



Examples of BaseDB subclasses: UserDB, LessonsDB, PaymentDB, NotificationDB, SupportTicketDB

תרשים 6: UML של המחלקות המרכזיות. החצים מצביעים למחלקת האב או לממשק.

פרויקט ViewDB

בפרויקט ViewDB נמצאות המחלקות שעובדות מול בסיס הנתונים. BaseDB אחראית על פתיחת החיבור ועל פעולות בסיסיות. מחלקות כמו UserDB, LessonsDB, PaymentDB ו-NotificationDB אחראיות כל אחת על טבלה או נושא מסוים.

קטע קוד: דוגמה לשליפה ולעדכון דרך LessonsDB.

```

public List<Lessons> GetAllStudentLessons(int sid)
{
    string sql = "SELECT * FROM [Lessons] WHERE StudentID = ?";
    return Select(sql, new OleDbParameter("@sid", sid))
        .OfType<Lessons>()
        .ToList();
}

public void CancelLesson(int lessonId)
{
    string sql = "UPDATE [Lessons] SET Canceled = 1 WHERE LessonId = ?";
    SaveChanges(sql, new OleDbParameter("@lessonId", lessonId));
}

```

פרויקט BusinessLogic

בפרויקט BusinessLogic נמצאות מחלקות עזר שמרכזות עבודה על רשימות ונתונים. לדוגמה, קיימות מחלקות עבור רשימת משתמשים, רשימת תלמידים ורשימת ערים. השכבה מיועדת ללוגיקה שלא שייכת למסך ולא לשאילתת SQL אחת.

שירות WCF

בחרתי בשירות WCF כדי שכל הלקוחות יעבדו מול אותו שרת. IService1 הוא הממשק שמגדיר אילו פעולות הלקוח יכול לבקש. Service1 מממש את הפעולות ומפנה אותן למחלקה המתאימה ב-ViewDB.

קטע קוד: דוגמאות לפעולות שהוגדרו בממשק IService1.

```

[OperationContract]
void AddLessonForStudent(int sid, string Date, string time);

[OperationContract]
List<Lessons> GetAllStudentLessons(int id);

[OperationContract]
List<Notification> GetUserNotifications(int userId, string userType);

```

קובצי App.config והגדרות הרשת

כתובת השירות לא כתובה בכל מסך בנפרד. היא נשמרת בקובצי App.config. בצד הלקוח מוגדרת הכתובת שאליה מתחברים, ובצד השרת מוגדרת כתובת הבסיס של השירות. אם כתובת השרת משתנה, מעדכנים אותה בקובץ ההגדרות.

קטע קוד: כתובת השירות בקובץ App.config.

```

<endpoint
    address="http://192.168.1.136:8733/Design_Time_Addresses/WcfServiceLibrary1/Service1/"
    binding="basicHttpBinding"
    contract="driver.IService1" />

<add baseAddress="http://192.168.1.136:8733/Design_Time_Addresses/WcfServiceLibrary1/Service1/"
/>

```

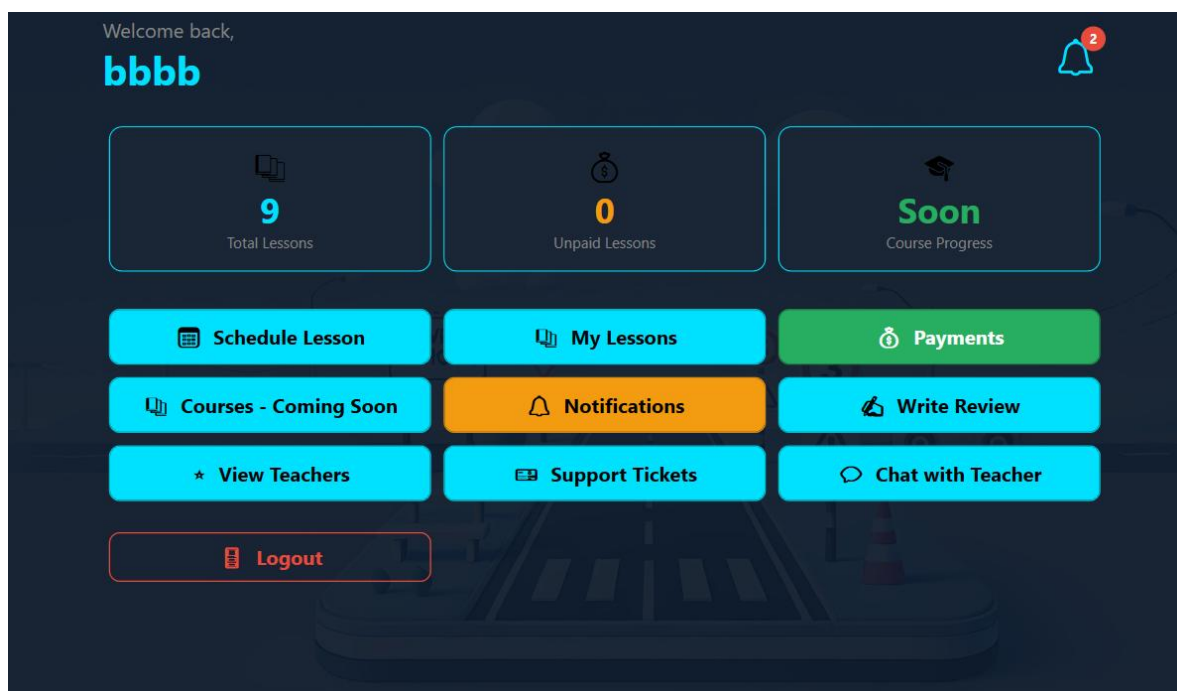
פרק 4: צד הלקוח

למערכת יש שלושה לקוחות. הם לא נועדו להחליף אחד את השני. כל לקוח מתאים למצב שימוש אחר, אבל כולם פונים לאותו שירות רשת ועובדים על אותם נתונים.

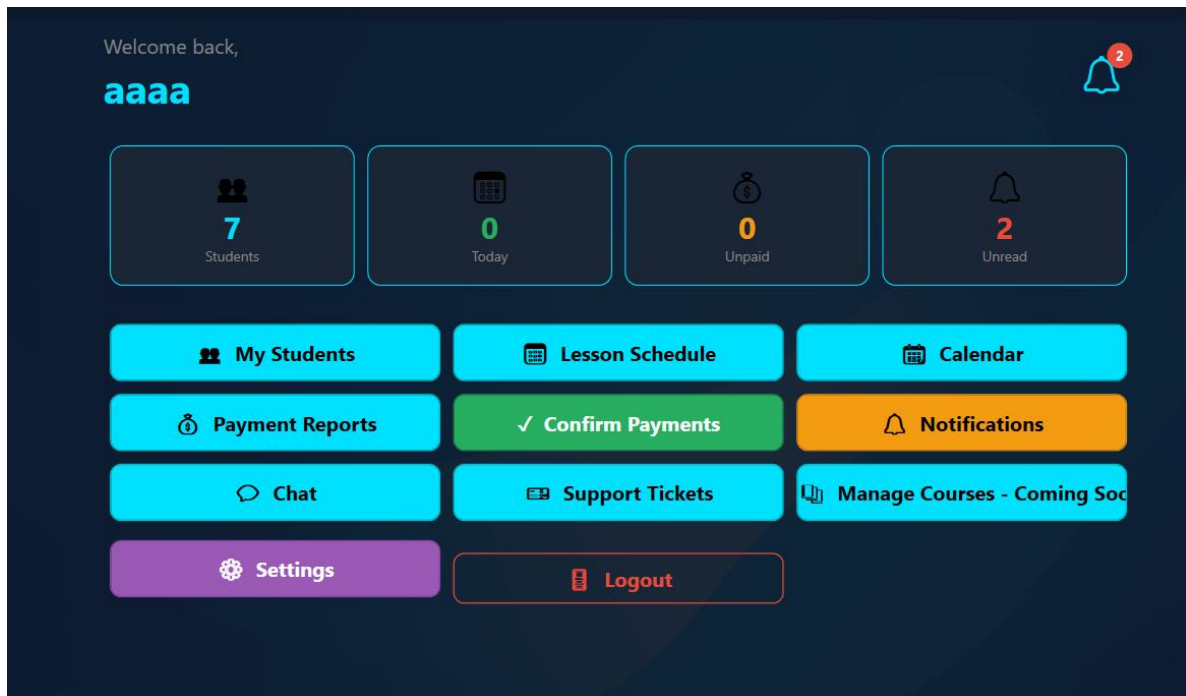
לקוח	מי משתמש בו	למה בחרתי בו
WPF	תלמיד, מורה ומנהל מהמחשב הביתי	נוח לעבודה מלאה עם מסכים גדולים והרבה פעולות.
Web	תלמיד, מורה או מנהל שנכנסים מדפדפן	לא דורש התקנה ומתאים לכניסה מהירה.
MAUI / Android	בעיקר תלמיד ומורה מהטלפון	מתאים לפעולות קצרות כמו צפייה, תשלום והתראות.

לקוח WPF

יישום WPF הוא הלקוח המרכזי למחשב. בחרתי בו כדי שתלמידים ומורים יוכלו לעבוד בנוחות מהבית. תלמיד יכול לקבוע שיעור, לראות תשלומים והתראות. מורה יכול לנהל תלמידים, לוח זמנים ודוחות.



תמונה 1: מסך הבית של תלמיד ביישום WPF.



תמונה 2: מסך הבית של מורה ביישום WPF.

Binding, XAML ו-ValueConverter

מסכי WPF נכתבו ב-XAML. בחלק מהמסכים השתמשתי ב-Binding כדי לחבר בין נתונים לפקדים. בנוסף קיימת המחלקה ImgConverter שמממשת IValueConverter וממירה מידע לצורה שאפשר להציג במסך.

קטע קוד: מחלקת הממיר בלקוח WPF.

```
public class ImgConverter : IValueConverter
{
    public object Convert(object value, Type targetType,
        object parameter, CultureInfo culture)
    {
        ...
    }
}
```

לקוח Web

לקוח Web נכתב עם Razor Pages. בחרתי בו כדי לאפשר כניסה מדפדפן בלי התקנה. יש בו דפים נפרדים לתלמיד, למורה ולמנהל, לדוגמה Notifications, Payments, ScheduleLesson ו-ManageUsers.

MAUI / Android לקוח

לקוח MAUI מיועד בעיקר לטלפון Android. בחרתי בו כדי לבצע פעולות קצרות גם כשלא נמצאים ליד מחשב. תלמיד יכול לצפות בשיעורים ולקבוע שיעור. מורה יכול לראות תשלומים והתראות.

Async / Tasks

בטלפון חשוב שהמסך לא ייתקע בזמן פנייה לשרת. לכן בלקוח MAUI השתמשתי בפעולות אסינכרוניות וב-await. המשתמש יכול להמשיך לקבל תגובה מהמסך בזמן שהבקשה נשלחת.

קטע קוד: קריאה אסינכרונית לשירות מתוך לקוח MAUI.

```
var lessons = await ServiceHelper.CallAsync(
    srv => srv.GetAllTeacherLessonsAsync(AppState.UserId));

await ServiceHelper.CallAsync(
    srv => srv.MarkLessonPaidAsync(lessonId));
```

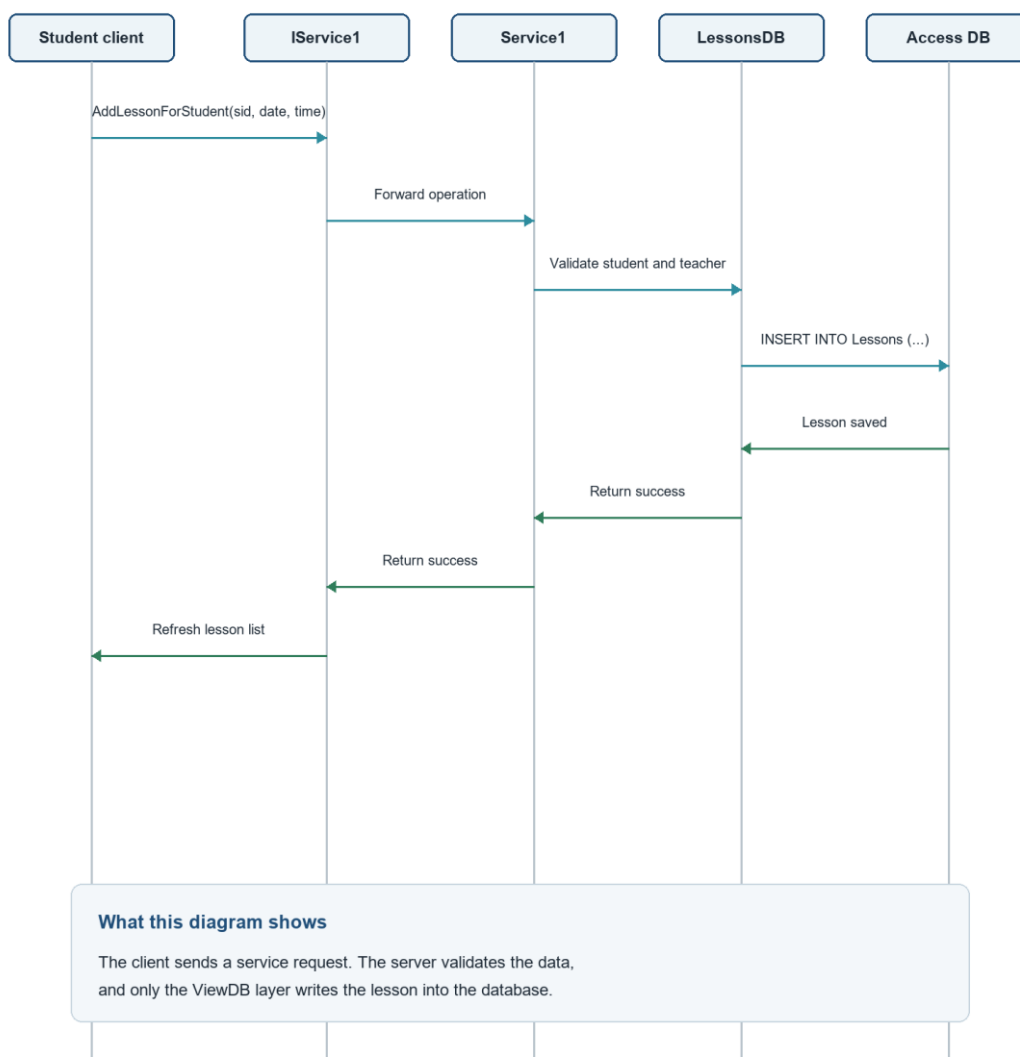
פרק 5: תהליכים מרכזיים ואבטחה

קביעת שיעור

כאשר תלמיד קובע שיעור, הלקוח שולח לשירות את מזהה התלמיד, התאריך והשעה. Service1 מעביר את הפעולה ל-LessonsDB. שם נבדק המורה של התלמיד ונשמרת רשומה חדשה בטבלת Lessons.

Sequence Diagram - Schedule a Lesson

Example of a complete request from a client screen to the Access database



תרשים 7: רצף הפעולות כאשר תלמיד קובע שיעור.

התראות

ההתראות נשמרות בטבלת Notifications. לכל התראה יש מזהה שולח, מזהה מקבל וסוג משתמש. כך השירות יודע להחזיר לכל משתמש רק את ההתראות שמתאימות לו. קיימות גם פעולות לסימון התראה כנקראה ולמחיקת התראה.

תשלומים ודוחות

פרטי התשלום נשמרים בטבלת Payments. המורה יכול לראות תשלומים לפי תלמיד, לאשר תשלום ולהפיק דוח לתקופה. התלמיד יכול לראות תשלומים פתוחים ולבדוק מה נשאר לשלם.

בדיקות קלט ואבטחה

בזמן הרשמה והתחברות אני לא שומר סיסמה רגילה במסד. הסיסמה עוברת גיבוב עם PBKDF2 ו-salt. בכניסה למערכת משווים בין הערך המחושב לבין הערך השמור.

בנוסף, השאילות משתמשות ב-OleDbParameter. כך הנתונים של המשתמש נשלחים כפרמטרים ולא מתחברים ישירות לטקסט של השאילתה. זה מקטין את הסיכון ל-SQL Injection.

קטע קוד: פרמטר בשאילתה וגיבוב סיסמה לפני שמירה.

```
string sql = "SELECT * FROM [Lessons] WHERE StudentID = ?";  
return Select(sql, new OleDbParameter("@sid", sid));  
  
string hashedPassword = SecurityHelper.HashPassword(user.Password);
```

פרק 6: מדריך הפעלה ובדיקות

הפעלה קצרה

1. פותחים את הפתרון WcfServiceLibrary1.sln בתוכנת Visual Studio.
2. מוודאים שקובץ UsersDataBase.accdb נמצא בתוך תיקיית ViewDB.
3. מריצים את שירות WCF.
4. מריצים את הלקוח הרצוי: Web, WPF או MAUI.
5. נכנסים עם משתמש מתאים ובודקים את הפעולה הרצויה.

בדיקות שביצעתי

תוצאה צפויה	מה בדקתי	בדיקה
מעבר למסך שמתאים לתפקיד.	שם משתמש וסיסמה תקינים	התחברות
כל משתמש רואה פעולות מתאימות.	תלמיד ומורה נכנסים למסכים שונים	בדיקת הרשאה
נשמר שיעור חדש במסד.	תלמיד בוחר תאריך ושעה	קביעת שיעור
מצב השיעור מתעדכן.	ביטול שיעור קיים	ביטול שיעור
מצב התשלום מתעדכן.	אישור תשלום על שיעור	תשלום
מוצגות רק התראות של המשתמש.	שליפה וסימון כנקרא	התראות
הפנייה נשמרת עם ההודעות.	פתיחת פנייה והוספת הודעה	פניות תמיכה

הערה לגבי מסכי הקורסים

בפרויקט קיימות טבלאות ומחלקות עבור קורסים, אבל חלק ממסכי הקורסים עדיין לא פתוחים למשתמש. ביישום WPF סימנתי אותם כ-Coming Soon כדי לא להציג מסך חלקי כאילו הוא מוכן.

פרק 7: נספחים

בגוף התיק הכנסתי רק קטעי קוד קצרים שעוזרים להבין את המימוש. קוד המקור המלא מצורף בסוף קובץ ההגשה המלא, אחרי העמוד האחרון של התיק.

קבצים שכדאי לפתוח בזמן הבחינה

- WcfServiceLibrary1/WcfServiceLibrary1/IService1.cs
- WcfServiceLibrary1/WcfServiceLibrary1/Service1.cs
- WcfServiceLibrary1/WcfServiceLibrary1/App.config
- WcfServiceLibrary1/ViewDB/BaseDB.cs
- WcfServiceLibrary1/ViewDB/LessonsDB.cs
- WcfServiceLibrary1/Model/Helpers/SecurityHelper.cs
- driver-client/driver-client/StudentUI.xaml
- driver-maui/Pages/LoginPage.xaml.cs

סיכום קצר

Driver Moodle מרכז את ניהול שיעורי הנהיגה במקום אחד. בניתי מערכת לקוח-שרת עם בסיס נתונים, שירות WCF ושלושה לקוחות. במהלך העבודה למדתי לחלק פרויקט לשכבות, לעבוד עם מסד נתונים ולהתאים ממשק משתמש לסוגים שונים של משתמשים.